Name: Yash Singh	Subject: Agile Software Development & DevOps
T.E. Batch: T22 Sem: 5th Roll No: 2401118	Date: 17/10/26

Course Code: 2165119 **Course Name:** ASD&D Lab

Class: TE **Batch:** T22 **Semester:** V

Student Name: Yash Singh **Roll No:** 2401118

—----------

EXPERIMENT NO. 01

Aim: - To perform version control operations using Git.

Theory:

A Version Control System (VCS) is a software tool that helps software teams manage changes to source code over time. It keeps track of every modification to the code in a special kind of database. If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while minimizing disruption to all team members.

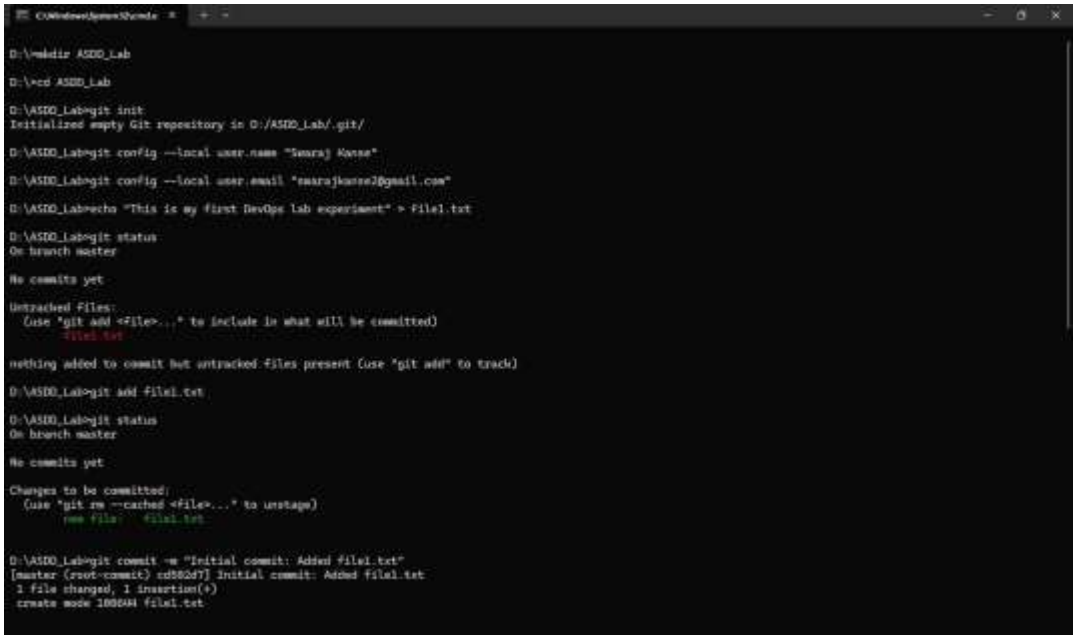
Git is a Distributed Version Control System (DVCS). Unlike centralized version control systems, Git gives every developer a local copy of the entire development history, making operations like branching, merging, and committing extremely fast and possible even ofline.

Basic Git Operations & Commands:

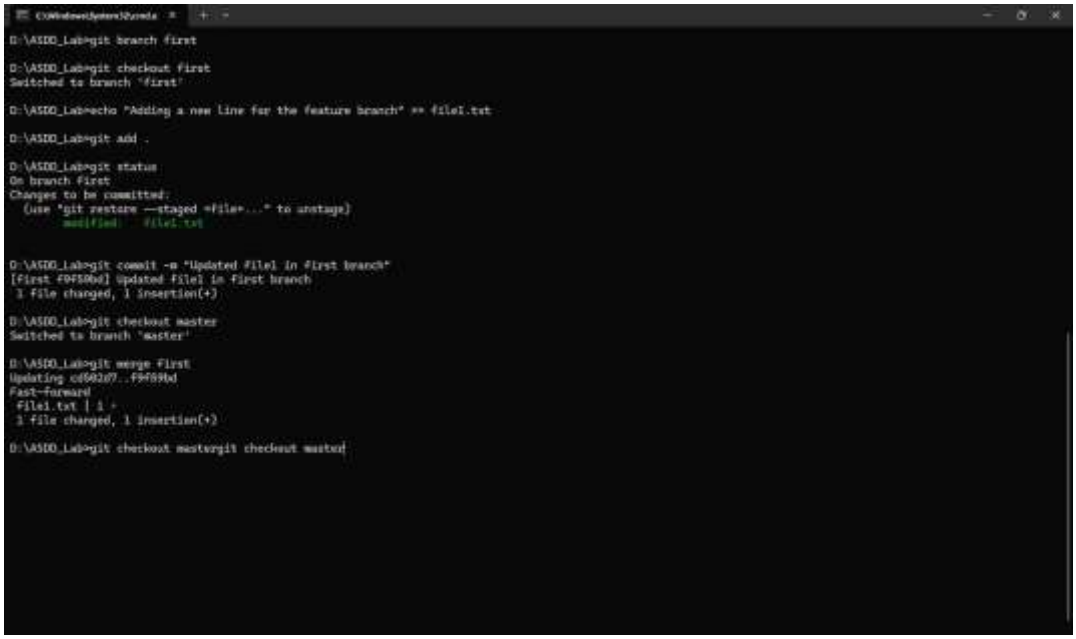
1. **Repository Initialization (git init)**
   Creates a new, empty Git repository or reinitializes an existing one. It creates a .git hidden folder that tracks all versions.
2. **Configuration (git config)**
   Used to set author details (name and email) that will be attached to your commits. Using --local ensures these details are only applied to the current repository.
3. **Checking Status (git status)**
   Displays the state of the working directory and the staging area. It lets you see which changes have been staged, which haven't, and which files aren't being tracked by Git.
4. **Staging Files (git add)**
   Adds a change in the working directory to the staging area. It tells Git that you want to include updates to a particular file in the next commit.
5. **Committing (git commit)**
   Captures a snapshot of the project's currently staged changes. Committed snapshots can be thought of as "safe" versions of a project that Git will never change unless you explicitly ask it to.
6. **Branching (git branch / git checkout)**
   Branches allow you to diverge from the main line of development and continue to do work without messing with that main line. git checkout is used to switch between these branches.
7. **Merging (git merge)**

Merges the specified branch's history into the current branch.

IMPLEMENTATION (Part 1: Initialization, Configuration, and First Commit)



IMPLEMENTATION (Part 2: Branching and Merging)



Conclusion: Thus, we have successfully studied the concepts of Version Control Systems and performed basic local repository operations including initialization, staging, committing, branching, and merging using Git.